

11月21日实验： 机器人模拟

一、 gazebo环境搭建

注意： 以下安装步骤适用于ROS2 Humble版本

Gazebo Fortress安装步骤

```
# 更多信息可参考官网教程: https://gazebo-sim.org/docs/harmonic/getstarted/

sudo apt-get update
sudo apt-get install lsb-release gnupg # 安装必要的软件包

# 安装Ignition Fortress
sudo curl https://packages.osrfoundation.org/gazebo.gpg --output /usr/share/keyrings/pkgs-osrf-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/pkgs-osrf-archive-keyring.gpg] \
https://packages.osrfoundation.org/gazebo/ubuntu-stable $(lsb_release -cs) \
main" | sudo tee /etc/apt/sources.list.d/gazebo-stable.list > /dev/null
sudo apt-get update
sudo apt-get install ignition-fortress

# 测试: 可以看到仿真GUI, 包含6个简单形状
ign gazebo shapes.sdf -v 4
```

ROS2 Humble TurtleBot3包安装步骤

```
#更多信息可参照官网教程: https://emanual.robotis.com/docs/en/platform/turtlebot3/quick-start/

# Install Cartographer
sudo apt install ros-humble-cartographer
sudo apt install ros-humble-cartographer-ros
# Install Navigation2
sudo apt install ros-humble-navigation2
sudo apt install ros-humble-nav2-bringup

# Install the required TurtleBot3 Packages
# source /opt/ros/humble/setup.bash # 安装ros时应当已经做过
mkdir -p ~/turtlebot3_ws/src #创建一个新的工作空间
cd ~/turtlebot3_ws/src/
# 如无法连接github可下载群文件中对应package放置于对应目录
git clone -b humble https://github.com/ROBOTIS-GIT/DynamixelSDK.git
git clone -b humble https://github.com/ROBOTIS-GIT/turtlebot3_msgs.git
git clone -b humble https://github.com/ROBOTIS-GIT/turtlebot3.git
sudo apt install python3-colcon-common-extensions # 安装ros时应当已经做过
cd ~/turtlebot3_ws
colcon build --symlink-install
echo 'source ~/turtlebot3_ws/install/setup.bash' >> ~/.bashrc
source ~/.bashrc

# Environment Configuration
echo 'export ROS_DOMAIN_ID=30 #TURTLEBOT3' >> ~/.bashrc
echo 'source /usr/share/gazebo/setup.sh' >> ~/.bashrc
# echo 'source /opt/ros/humble/setup.bash' >> ~/.bashrc 这一步安装ros时应当已经做过
source ~/.bashrc
```

```
# Install the required TurtleBot3 Simulation Packages
cd ~/turtlebot3_ws/src/
git clone -b humble https://github.com/ROBOTIS-GIT/turtlebot3_simulations.git
cd ~/turtlebot3_ws && colcon build --symlink-install
# 测试：空世界+小车
export TURTLEBOT3_MODEL=burger
ros2 launch turtlebot3_gazebo empty_world.launch.py
```

二、WanderBot实现

参照教材思路使用ROS2和Gazebo Sim实现WanderBot的仿真模拟，要求编写节点读取小车的雷达数据并对小车的速度进行控制，实现小车的雷达避障功能。

可以参考教材小车基本的避障逻辑：无障碍则直行，有障碍则停下并转向。同学们可以自行完善避障逻辑以实现更好的效果。

注意点：

1. 使用ros2 topic工具和interface工具查看小车速度话题 **/cmd_vel** 和雷达数据话题 **/scan** 的详细信息
2. **代码参考**

```

# avoidance.py
#ros lib
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import TwistStamped, Twist
from sensor_msgs.msg import LaserScan

#commom lib
import math
import numpy as np
print ("improt done")
RAD2DEG = 180 / math.pi

class laserAvoid(Node):
    def __init__(self,name):
        super().__init__(name)
        #订阅雷达数据话题
        self.sub_laser = self.create_subscription(LaserScan,"/scan",self.registerScan,1)
        #发布小车速度话题
        self.pub_vel = self.create_publisher(TwistStamped,'/cmd_vel',1)

        #设置参数
        self.declare_parameter("linear",0.3) #线速度
        self.linear = self.get_parameter('linear').get_parameter_value().double_value
        self.declare_parameter("angular",1.0) #角速度
        self.angular = self.get_parameter('angular').get_parameter_value().double_value
        self.declare_parameter("LaserAngle",45.0) #雷达避障检测角度(-LaserAngle, LaserAngle)
        self.LaserAngle = self.get_parameter('LaserAngle').get_parameter_value().double_value
        self.declare_parameter("ResponseDist",0.55) #避障响应距离
        self.ResponseDist = self.get_parameter('ResponseDist').get_parameter_value().double_value

# 停止小车
def exit_pro(self):
    ts = TwistStamped()
    ts.header.stamp = self.get_clock().now().to_msg()
    ts.twist.linear.x = 0.0
    ts.twist.angular.z = 0.0
    self.pub_vel.publish(ts)

# TODO 雷达数据回调函数，实现小车的避障逻辑
# 可以自由修改本函数和其他参数，或添加任何其他需要的函数和类变量，提供的代码仅作参考和提示用
def registerScan(self, scan_data):
    if not isinstance(scan_data, LaserScan): return
    ranges = np.array(scan_data.ranges)
    for i in range(len(ranges)):
        # 获得当前距离数据对应的角度
        # angle = (scan_data.angle_min + scan_data.angle_increment * i) * RAD2DEG
        pass

    # 在这里利用雷达的距离数据，实现避障逻辑

def main():
    rclpy.init()
    laser_avoid = laserAvoid("laser_Avoidance")
    print ("laser avoidance started")
    try:

```

```
    rclpy.spin(laser_avoid)
except KeyboardInterrupt:
    pass
finally:
    laser_avoid.exit_pro()
    laser_avoid.destroy_node()
```

使用的仿真环境：

```
$ export TURTLEBOT3_MODEL=waffle
$ ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

提交要求：

提交实验报告（包含实现思路和结果截图）以及代码（包含控制小车行动逻辑的整个节点代码即可）